МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №10

з дисципліни «Професійний Пайтон»

на тему: «Production Python: Docker, CI/CD та захист проєкту»

Виконав:
студент групи ФеП-32
Накладюк Володимир Тарасович
Варіант №15

Львів - 2026

1. Мета роботи

Підготувати Python-проєкт до production-oriented використання: оформити його як пакет (wheel і sdist), винести конфігурацію в змінні середовища без збереження секретів у коді, створити Docker image і запустити контейнер, налаштувати CI/CD pipeline у GitHub Actions з автоматичними lint, type checking, тестами, збиранням пакета і Docker-образу, оновити документацію та підготувати проєкт до фінального захисту.

2. Індивідуальне завдання

Варіант 15. Облік результатів наукових експериментів.

- продовжити проєкт Lab_01 (назва експерименту, дослідник, параметр, результат, дата) і зберегти функціональність: додавання, пошук за дослідником, average/min/max, фільтр за діапазоном, сортування за датою;
- production package/module: `experiment-registry` / `experiment_registry`;
- зберегти і задокументувати REST API з Lab_08 (`/researchers`, `/experiments`, `/researchers/{id}/experiments`, `/experiments/statistics`), додати `GET /health`;
- використати persistence layer Lab_07 і оптимізації Lab_09; конфігурацію передавати через `DATABASE_URL`, `APP_ENV`/`ENVIRONMENT`, `LOG_LEVEL`; секрети не зберігати в коді та Git;
- pyproject.toml metadata, wheel і sdist, перевірка встановлення wheel у чистому venv;
- Dockerfile, .dockerignore, image, container, persistent volume для SQLite;
- GitHub Actions: ruff check, format check, mypy, pytest, package build і docker build;
- README: предметна система, встановлення, конфігурація, локальний і Docker-запуск, API, тести, CI/CD, демонстрація.

3. Короткий опис проєкту та фінальна структура

experiment-registry — система обліку результатів наукових експериментів, яка розвивалася протягом курсу: консольний застосунок і структури даних (ЛР 1–2), потокова обробка (ЛР 3), типізована доменна модель (ЛР 4), надійний імпорт/експорт (ЛР 5), test suite (ЛР 6), SQLite з SQLAlchemy та Alembic (ЛР 7), асинхронний REST API (ЛР 8) і оптимізована статистика (ЛР 9). У лабораторній №10 додано конфігурацію через змінні середовища, `/health`, packaging, Docker і CI.

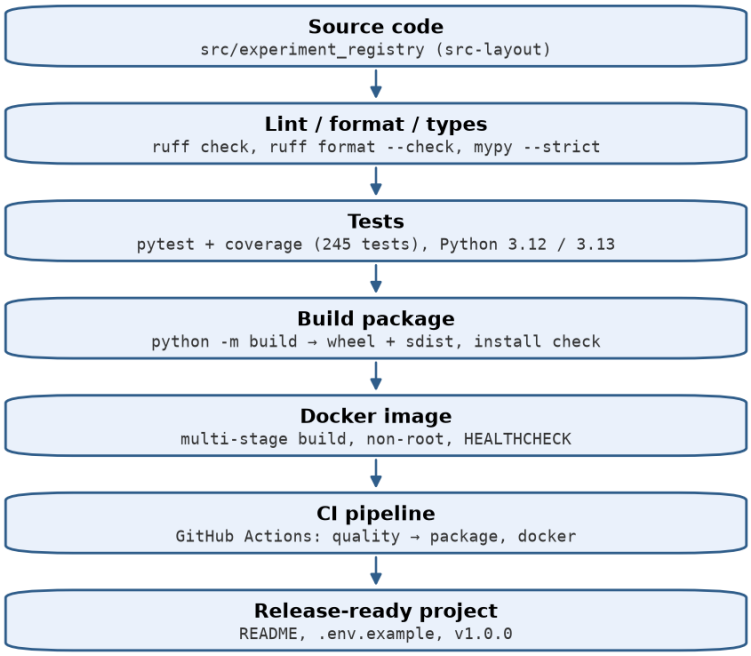


Рис. 10.1. Production lifecycle проєкту

Лістинг 10.1. Фінальна структура проєкту (без вкладених пакетів попередніх ЛР)

```
lab10/
├── benchmarks/
│   ├── benchmark_performance.py
│   ├── benchmark_search.py
│   ├── benchmark_streaming.py
│   └── import_experiments.py
├── migrations/
│   ├── versions/
│   │   ├── 0001_create_researchers_and_experiments.py
│   │   └── 0002_add_experiment_notes.py
│   ├── env.py
│   └── script.py.mako
├── src/
│   └── experiment_registry/
│       ├── __init__.py
│       ├── analytics.py
│       ├── config.py
│       ├── data.py
│       ├── decorators.py
│       ├── exceptions.py
│       ├── logging_setup.py
│       ├── main.py
│       ├── models.py
│       ├── processors.py
│       ├── py.typed
│       ├── services.py
│       └── settings.py
├── tests/
│   └── conftest.py
├── .dockerignore
├── .env.example
├── .gitignore
├── alembic.ini
├── config.yaml
├── Dockerfile
├── LICENSE
├── pyproject.toml
└── README.md

.github/
└── workflows/
    └── lab10-ci.yml
```

4. Хід виконання

4.1. pyproject.toml і package metadata

Метадані пакета описано в секції `[project]`: назва, версія 1.0.0 (semantic versioning), опис, README, мінімальна версія Python, ліцензія MIT з файлом LICENSE, автор, ключові слова, classifiers, залежності й посилання на репозиторій. Додатковий набір `[dev]` містить інструменти розробки. `py.typed` повідомляє type checker-ам, що пакет типізований. Конфігурація ruff, mypy, pytest і coverage зберігається в тому самому файлі.

Лістинг 10.2. pyproject.toml

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "experiment-registry"
version = "1.0.0"
description = "Registry of scientific experiment results: REST API, persistence, import/export and
analytics"
readme = "README.md"
requires-python = ">=3.11"
license = "MIT"
license-files = ["LICENSE"]
authors = [{ name = "Volodymyr Nakladiuk" }]
keywords = ["experiments", "research", "fastapi", "sqlalchemy", "education"]
classifiers = [
    "Development Status :: 5 - Production/Stable",
    "Framework :: FastAPI",
    "Intended Audience :: Education",
    "Intended Audience :: Science/Research",
    "Operating System :: OS Independent",
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
    "Programming Language :: Python :: 3.13",
    "Typing :: Typed",
]
dependencies = [
    "PyYAML>=6.0",
    "SQLAlchemy>=2.0",
    "alembic>=1.13",
    "fastapi>=0.110",
    "uvicorn>=0.29",
    "httpx>=0.27",
    "pydantic>=2.6",
    "pydantic-settings>=2.2",
    "numpy>=1.26",
]

[project.optional-dependencies]
dev = [
    "mypy>=1.10",
    "types-PyYAML",
    "pytest>=8.0",
    "pytest-cov>=5.0",
    "pytest-asyncio>=0.23",
    "httpx2",  # transport required by starlette.testclient in current Starlette releases
    "ruff>=0.5",
    "build>=1.2",
]

[project.urls]
Repository = "https://github.com/3efirok/nakladiuk-lab-works"
Documentation = "https://github.com/3efirok/nakladiuk-lab-works/tree/main/lab10#readme"

[project.scripts]
experiment-registry = "experiment_registry.main:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.setuptools.package-data]
experiment_registry = ["py.typed"]
```

```
[tool.ruff]
line-length = 110
target-version = "py311"
extend-exclude = ["migrations/versions"]

[tool.ruff.lint]
select = ["E", "W", "F", "I", "B", "UP", "SIM", "C4"]

[tool.ruff.lint.per-file-ignores]
```

4.2. Build wheel і sdist

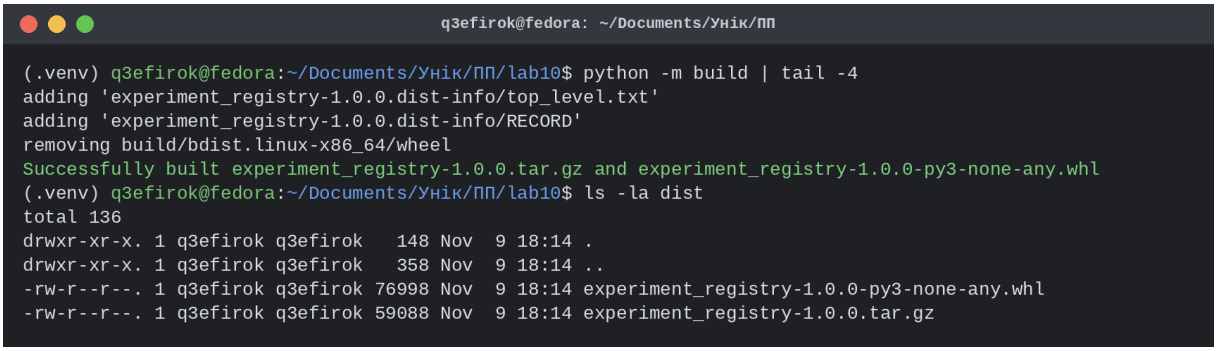


Рис. 10.2. Збирання пакета: python -m build

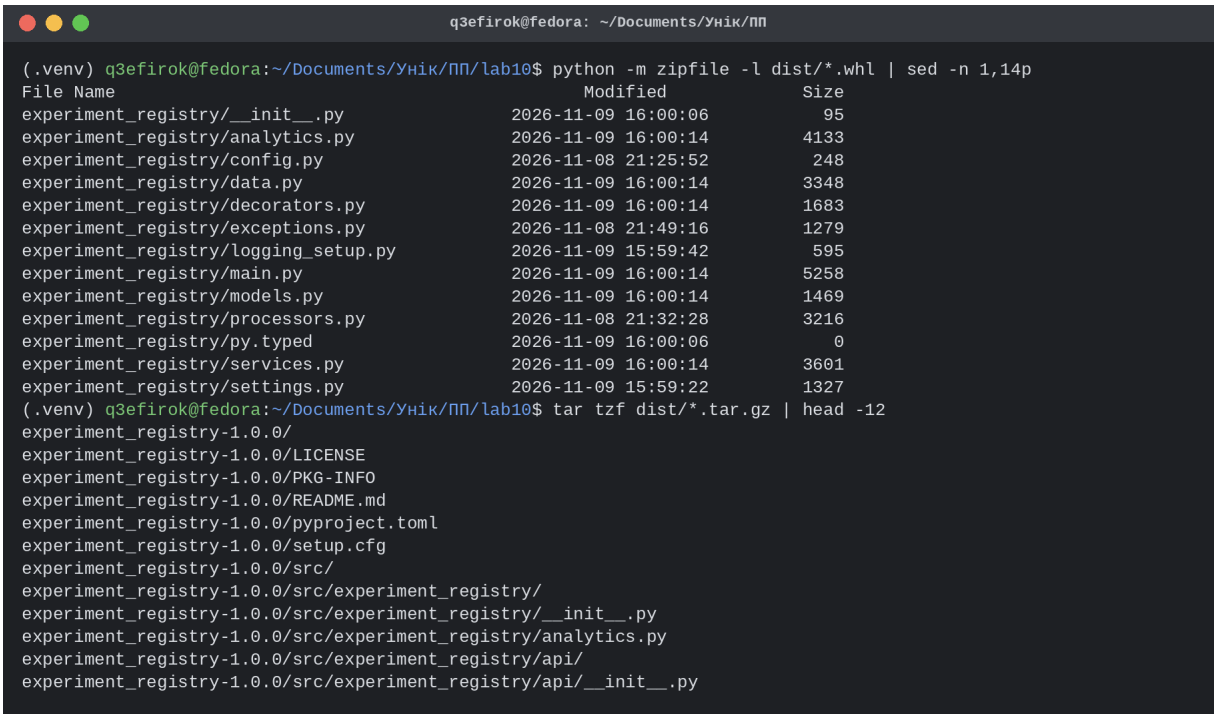


Рис. 10.3. Вміст wheel і sdist

Wheel — готовий до встановлення архів з кодом і метаданими (без кроку збирання), sdist — архів вихідного коду з pyproject.toml, README і LICENSE, з якого wheel можна зібрати знову.

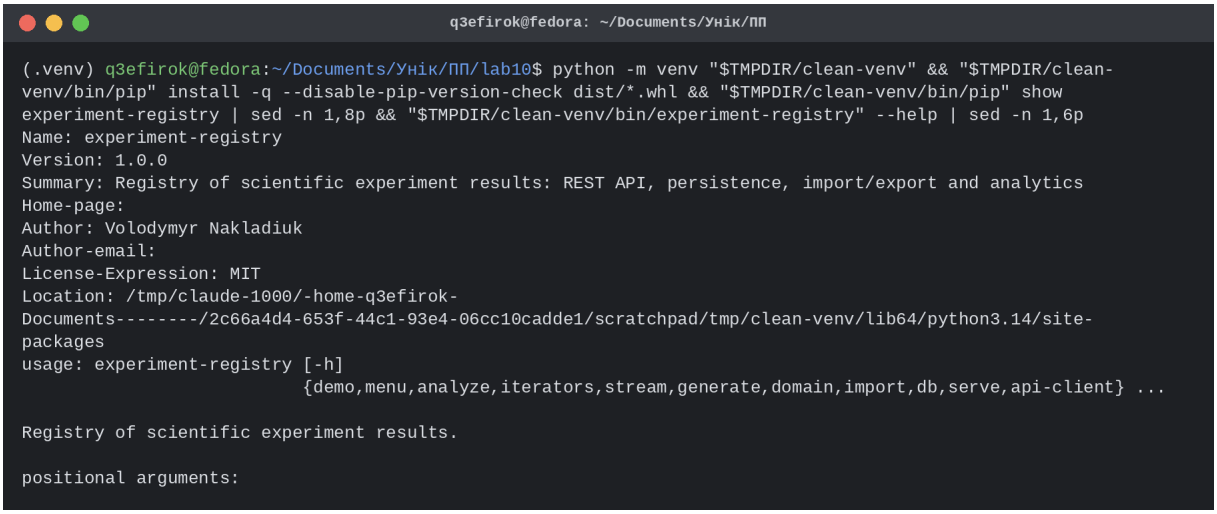


Рис. 10.4. Встановлення wheel у чистому віртуальному середовищі

4.3. Environment variables, .env.example і config

Налаштування описано класом `Settings` на pydantic-settings: значення читаються зі змінних середовища (а локально — з файлу `.env`), перевіряються за типами (наприклад, `ENVIRONMENT` лише development/test/production) і кешуються функцією `get_settings()`. Змінна оточення `ENVIRONMENT` має аліас `APP_ENV`.

Лістинг 10.3. settings.py (config)

```
"""Application settings from environment variables (12-factor configuration).

Values are read from the process environment and, for local development, from a `.env`
file that is never committed to Git. Secrets use `SecretStr`, so they are not printed
in logs, reprs or tracebacks.
"""

from functools import lru_cache
from typing import Literal

from pydantic import AliasChoices, Field, SecretStr
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8",
extra="ignore")

    app_name: str = "Experiment Registry"
    environment: Literal["development", "test", "production"] = Field(
        default="development", validation_alias=AliasChoices("ENVIRONMENT", "APP_ENV")
    )
    database_url: str = "sqlite:///experiments.db"
    log_level: Literal["DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"] = "INFO"
    api_token: SecretStr | None = Field(
        default=None, description="If set, POST/PATCH/DELETE require the X-API-Token
header."
    )

    @property
    def is_production(self) -> bool:
        return self.environment == "production"


@lru_cache
def get_settings() -> Settings:
    """Settings are created once per process; tests call get_settings.cache_clear()."""
    return Settings()
```

Лістинг 10.4. Файл .env.example

```
# Copy to .env for local development. .env is ignored by Git - never commit real secrets.
APP_NAME="Experiment Registry"
ENVIRONMENT=development            # development | test | production (alias: APP_ENV)
DATABASE_URL=sqlite:///experiments.db
LOG_LEVEL=INFO                     # DEBUG | INFO | WARNING | ERROR | CRITICAL
# API_TOKEN=<long random value>    # optional; protects POST/PATCH/DELETE (X-API-Token
header)
```

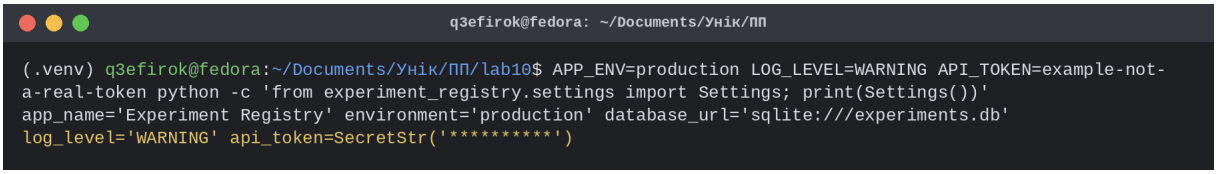


Рис. 10.5. Значення з оточення; секрет у repr прихований

4.4. Робота із secrets

Реальні паролі, токени та ключі не зберігаються в коді, README, Dockerfile або Git: файл `.env` додано до `.gitignore` і `.dockerignore`, у репозиторії є лише `.env.example` без справжніх значень. Необов'язковий `API_TOKEN` (захист POST/PATCH/DELETE заголовком `X-API-Token`) має тип `SecretStr`, тому не потрапляє в логи, repr і traceback. У CI секрети, якби вони знадобилися, передаються через GitHub Secrets, а в контейнер — через `-e` або `--env-file`.

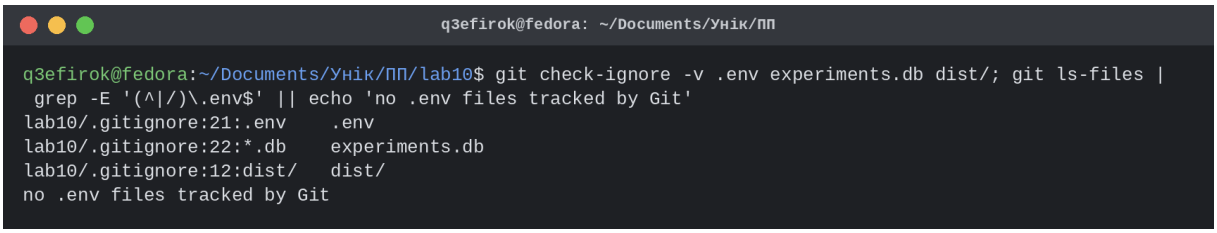


Рис. 10.6. Перевірка, що .env, база і артефакти збирання не потрапляють у Git

Лістинг 10.5. Перевірка токена для операцій запису

```
def require_token(request: Request, x_api_token: Annotated[str | None, Header()] = None)
-> None:
    """Write operations need the X-API-Token header when API_TOKEN is configured."""
    settings: Settings = request.app.state.settings
    expected = settings.api_token
    if expected is not None and x_api_token != expected.get_secret_value():
        raise HTTPException(status_code=401, detail="Missing or invalid X-API-Token
header")


WriteAccess = [Depends(require_token)]
```

4.5. Health endpoint і production logging

Лістинг 10.6. GET /health з перевіркою бази даних

```
    @app.get("/health", tags=["service"], responses={503: {"description": "Database
unavailable"}})
    def health(response: Response) -> dict[str, str]:
        """Liveness/readiness probe for Docker HEALTHCHECK and orchestration."""
        try:
            with engine.connect() as connection:
                connection.execute(text("SELECT 1"))
            database = "ok"
```

```
        except Exception:  # noqa: BLE001 - report any database failure as unavailable
            database = "unavailable"
            response.status_code = status.HTTP_503_SERVICE_UNAVAILABLE
        return {
            "status": "ok" if database == "ok" else "degraded",
            "database": database,
            "version": __version__,
            "environment": settings.environment,
        }
```

Лістинг 10.7. Налаштування журналювання за LOG_LEVEL

```
"""Production logging: one format for the application and uvicorn, level from
LOG_LEVEL."""

import logging
import logging.config

FORMAT = "%(asctime)s %(levelname)-8s %(name)s: %(message)s"


def configure_logging(level: str = "INFO") -> None:
    logging.config.dictConfig(
        {
            "version": 1,
            "disable_existing_loggers": False,
            "formatters": {"default": {"format": FORMAT}},
            "handlers": {"console": {"class": "logging.StreamHandler", "formatter":
"default"}},
            "root": {"handlers": ["console"], "level": level},
        }
    )
```

4.6. Dockerfile і .dockerignore

Dockerfile багатоетапний: на першому етапі зі src збирається wheel, на другому в чистий образ `python:3.12-slim` встановлюється лише wheel, без інструментів збирання й тестів. Застосунок працює від непривілейованого користувача `app`, база SQLite лежить у томі `/data`, значення за замовчуванням задано через `ENV` (без секретів). HEALTHCHECK викликає `/health`, а команда запуску спочатку застосовує міграції Alembic, потім запускає uvicorn.

Лістинг 10.8. Dockerfile

```
# syntax=docker/dockerfile:1

# ---------- stage 1: build the wheel ----------
FROM python:3.12-slim AS builder
WORKDIR /build
RUN pip install --no-cache-dir build
COPY pyproject.toml README.md LICENSE ./
COPY src ./src
RUN python -m build --wheel --outdir /dist

# ---------- stage 2: runtime image ----------
FROM python:3.12-slim AS runtime
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    ENVIRONMENT=production \
    LOG_LEVEL=INFO \
    DATABASE_URL=sqlite:////data/experiments.db

RUN useradd --create-home --uid 10001 app \
    && mkdir /data && chown app:app /data
WORKDIR /app

COPY --from=builder /dist/*.whl /tmp/
RUN pip install --no-cache-dir /tmp/*.whl && rm /tmp/*.whl
```

```
COPY --chown=app:app alembic.ini ./
COPY --chown=app:app migrations ./migrations

USER app
VOLUME ["/data"]
EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD python -c "import sys, urllib.request;
sys.exit(urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=4).status != 200)"

# apply migrations, then start the API
CMD ["sh", "-c", "alembic upgrade head && exec uvicorn experiment_registry.api:app --host
0.0.0.0 --port 8000"]
```

Лістинг 10.9. .dockerignore

```
# Keep the build context small and free of secrets
.git
.env
.venv/
venv/
**/__pycache__/
*.pyc
.pytest_cache/
.mypy_cache/
.ruff_cache/
.coverage
htmlcov/
dist/
build/
*.egg-info/
*.db
*.pdf
data/generated/
benchmarks/results/
output/
logs/
```

4.7. Docker build і Docker image

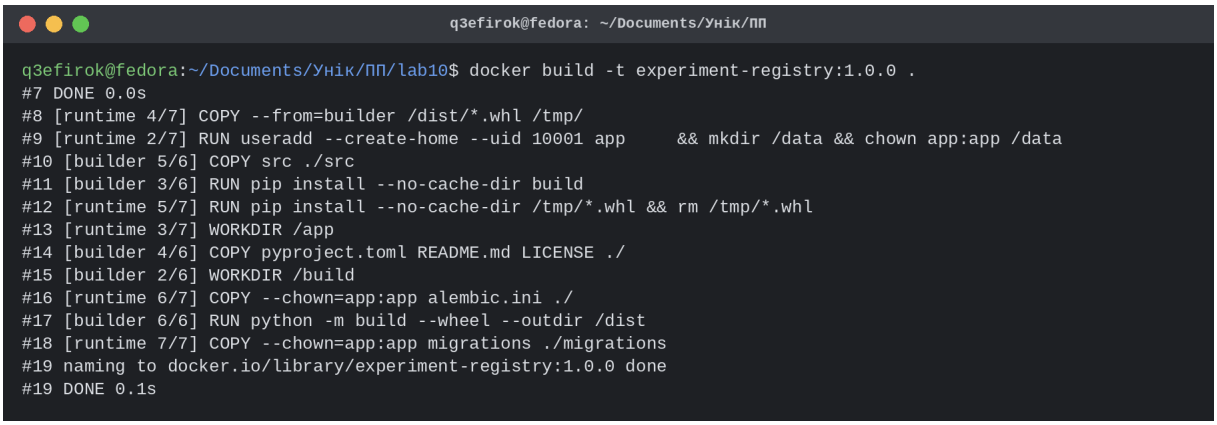


Рис. 10.7. Команда docker build (етапи збирання)

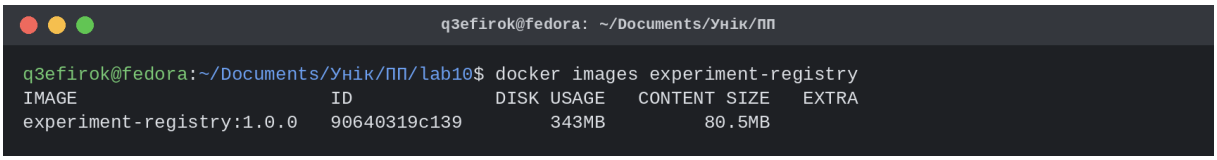


Рис. 10.8. Створений Docker image

4.8. Docker run, running container і /health

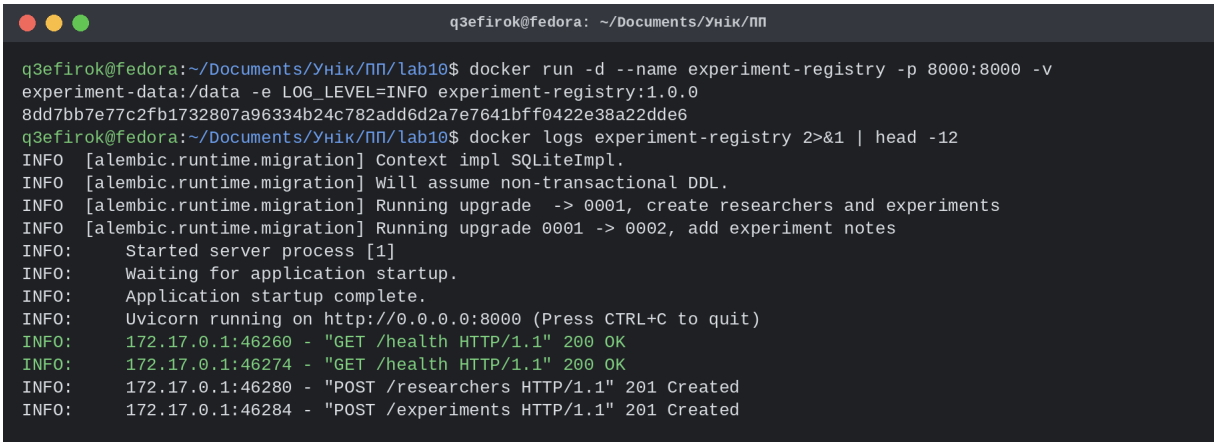


Рис. 10.9. Запуск контейнера з томом і змінною середовища; журнал старту

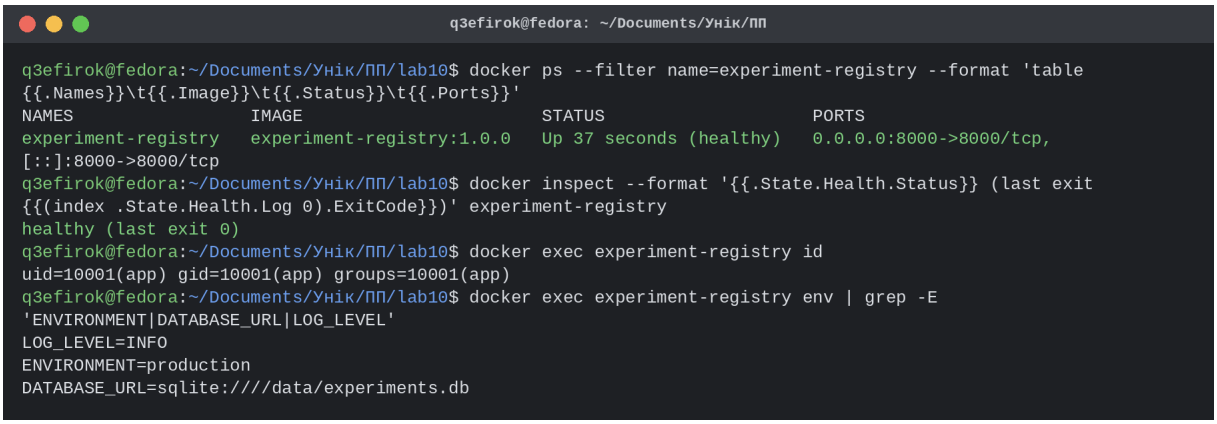


Рис. 10.10. Запущений контейнер: стан healthy, користувач, змінні середовища



Рис. 10.11. Запити до API в контейнері: /health, додавання даних, статистика

```
q3efirok@fedora: ~/Documents/Унік/ПП
q3efirok@fedora:~/Documents/Унік/ПП/lab10$ docker rm -f experiment-registry
experiment-registry
q3efirok@fedora:~/Documents/Унік/ПП/lab10$ docker run -d --name experiment-registry -p 8000:8000 -v
experiment-data:/data experiment-registry:1.0.0
1b96cfdc38399df86d257e53ef58d6c83bbd0d49fe18f91b01af8efcac78bf3e
q3efirok@fedora:~/Documents/Унік/ПП/lab10$ curl -s
'http://127.0.0.1:8000/experiments/statistics?parameter=temperature'
{"count":2,"average":297.6,"minimum":293.4,"maximum":301.8,"parameter":"temperature","researcher_id":null}
```

Рис. 10.12. Після перезапуску контейнера дані збереглися в томі experiment-data

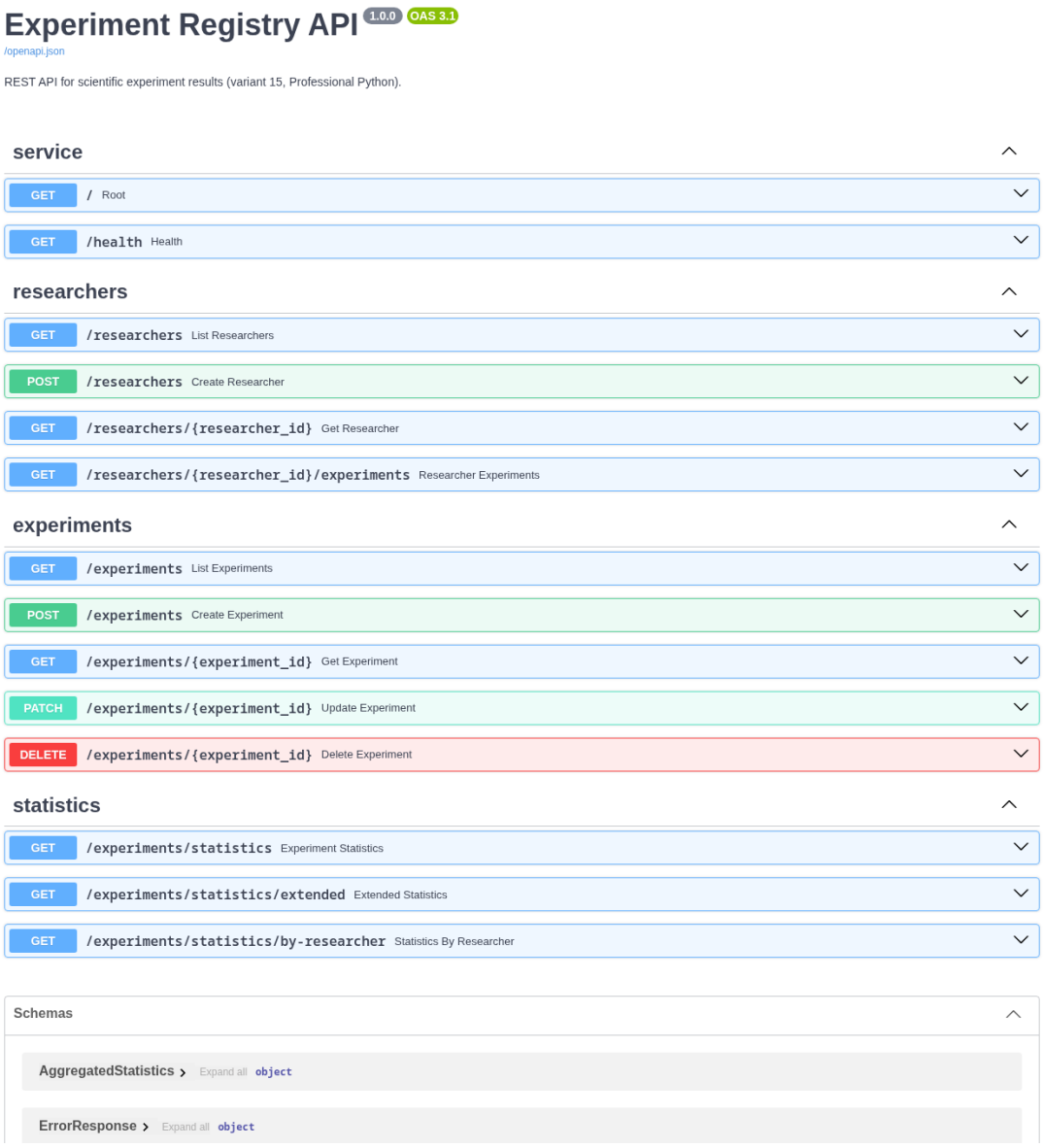


Рис. 10.13. Swagger UI застосунку, запущеного в Docker-контейнері

4.9. GitHub Actions workflow

Workflow розміщено в `.github/workflows/lab10-ci.yml` у корені репозиторію, бо GitHub запускає workflows лише звідти; `defaults.run.working-directory: lab10` і фільтр `paths` обмежують його проєктом лабораторної №10. Job `quality` (матриця Python 3.12 і 3.13) виконує ruff check, ruff format --check, mypy і pytest з coverage; після нього паралельно працюють `package` (wheel/sdist,

перевірка встановлення, artifact) і `docker` (docker build і перевірка `/health` у запущеному контейнері).

Лістинг 10.10. Workflow lab10-ci.yml

```
name: CI lab10 (experiment-registry)

on:
  push:
    branches: [main]
    paths: ["lab10/**", ".github/workflows/lab10-ci.yml"]
  pull_request:
    paths: ["lab10/**", ".github/workflows/lab10-ci.yml"]
  workflow_dispatch:

defaults:
  run:
    working-directory: lab10

jobs:
  quality:
    name: Lint, types, tests (Python ${{ matrix.python-version }})
    runs-on: ubuntu-24.04
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.12", "3.13"]
    steps:
      - name: Checkout
        uses: actions/checkout@v5
      - name: Setup Python
        uses: actions/setup-python@v6
        with:
          python-version: ${{ matrix.python-version }}
          cache: pip
          cache-dependency-path: lab10/pyproject.toml
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -e ".[dev]"
      - name: Lint (ruff check)
        run: ruff check .
      - name: Format check (ruff format)
        run: ruff format --check .
      - name: Type check (mypy)
        run: mypy
      - name: Tests with coverage
        run: pytest -v --cov --cov-report=term-missing

  package:
    name: Build wheel and sdist
    needs: quality
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-python@v6
        with:
          python-version: "3.12"
      - name: Build
        run: |
          python -m pip install --upgrade pip build
          python -m build
      - name: Check dist
        run: ls -la dist
      - name: Install wheel in a clean virtual environment
        run: |
          python -m venv /tmp/clean
          /tmp/clean/bin/pip install dist/*.whl
          /tmp/clean/bin/experiment-registry --help
      - name: Upload artifacts
        uses: actions/upload-artifact@v6
        with:
          name: experiment-registry-dist
          path: lab10/dist/

  docker:
    name: Docker build and smoke test
    needs: quality
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v5
      - name: Docker build
        run: docker build -t experiment-registry:ci .
```

```
      - name: Run container and check /health
        run: |
          docker run -d --name api -p 8000:8000 experiment-registry:ci
          for i in $(seq 1 30); do
            if curl -sf http://127.0.0.1:8000/health; then echo; exit 0; fi
            sleep 1
          done
          docker logs api
          exit 1
```

4.10. Lint, mypy і pytest

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab10$ ruff check . && ruff format --check .
All checks passed!
105 files already formatted
```

Рис. 10.14. ruff check і ruff format --check

Перед підключенням CI `ruff` виявив 61 зауваження, серед них 20 випадків B023: лямбди в циклах benchmark-скриптів захоплювали змінну циклу. Вони виконувалися одразу, тому результати були правильними, але такий код легко зламати при рефакторингу — лямбди замінено на `functools.partial`. Код відформатовано `ruff format`.

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab10$ mypy
Success: no issues found in 63 source files
```

Рис. 10.15. mypy --strict

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab10$ pytest -q --cov | tail -14
src/experiment_registry/settings.py                      17      0      0      0   100%
src/experiment_registry/streaming/__init__.py             1      0      0      0   100%
src/experiment_registry/streaming/analytics.py           34      0     12      0   100%
src/experiment_registry/streaming/datagen.py             24     16      4      0    29%   23-45
src/experiment_registry/streaming/eager.py               22     22      4      0     0%   3-30
src/experiment_registry/streaming/filters.py             46      2     14      2    93%   30, 55
src/experiment_registry/streaming/iterators.py           46      1     12      1    97%   54, 75->exit
src/experiment_registry/streaming/parsers.py             21      0      8      0   100%
src/experiment_registry/streaming/pipeline.py            31      0      8      0   100%
src/experiment_registry/streaming/readers.py             12      1      0      0    92%   23
-----------------------------------------------------------------------------------------------
TOTAL                                                  2596    145    482     60    93%
Required test coverage of 80.0% reached. Total coverage: 92.88%
245 passed in 4.49s
```

Рис. 10.16. pytest з coverage

Під час підготовки виявлено і виправлено дефект ізоляції тестів: `get_settings()` кешується, тому тест, що підміняв `DATABASE_URL`, використовував закешований URL і створював `experiments.db` у каталозі проєкту; тепер тест очищує кеш налаштувань. Для стабільності на повільніших CI-машинах збільшено запас часу в тесті паралельної статистики, а ProcessPoolExecutor явно використовує `forkserver`/`spawn`, бо `fork()` у багатопотоковому процесі небезпечний.

4.11. Результати CI: lint, тести, package build і Docker build

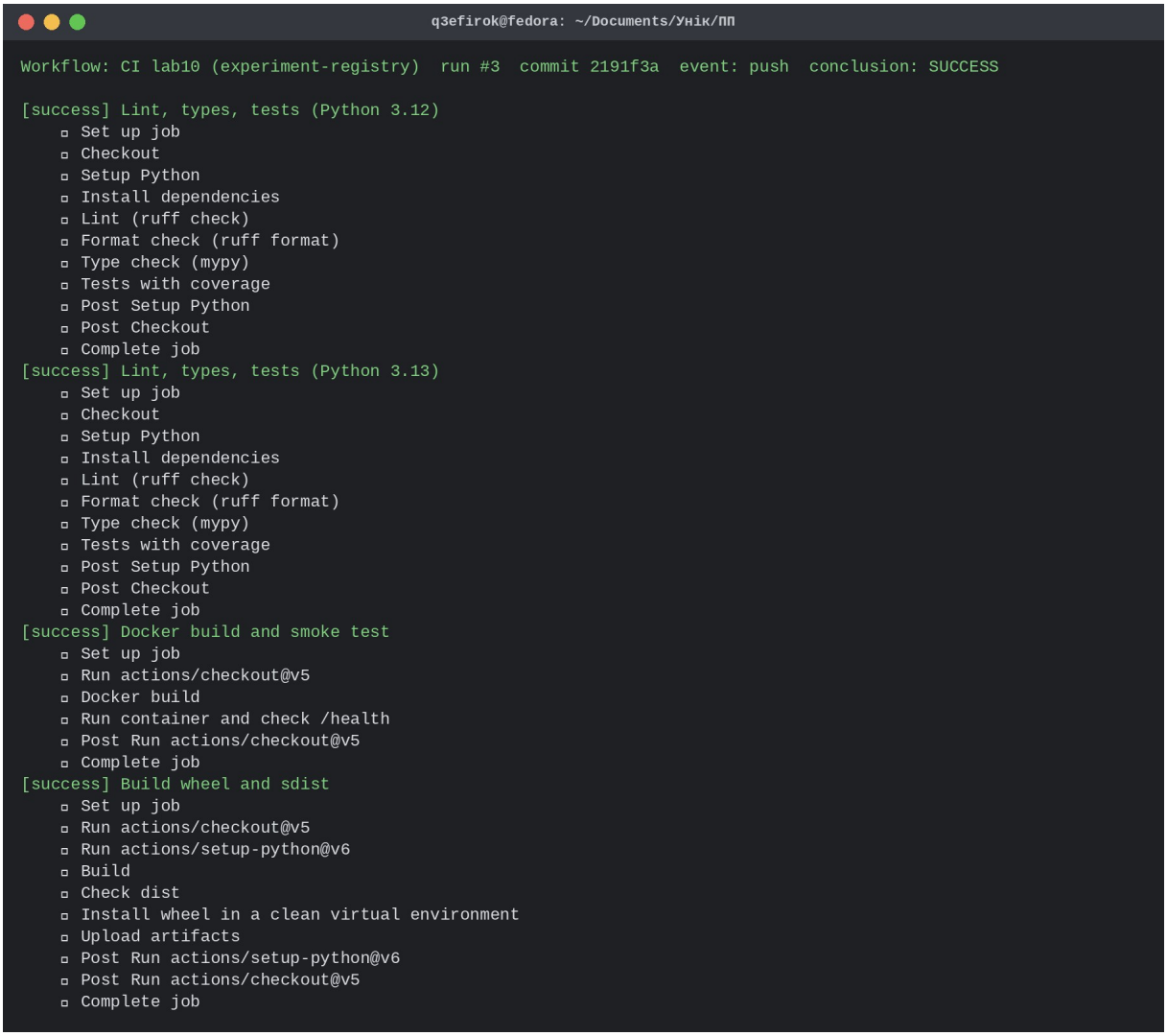


Рис. 10.17. Jobs і кроки запуску GitHub Actions (дані GitHub API)

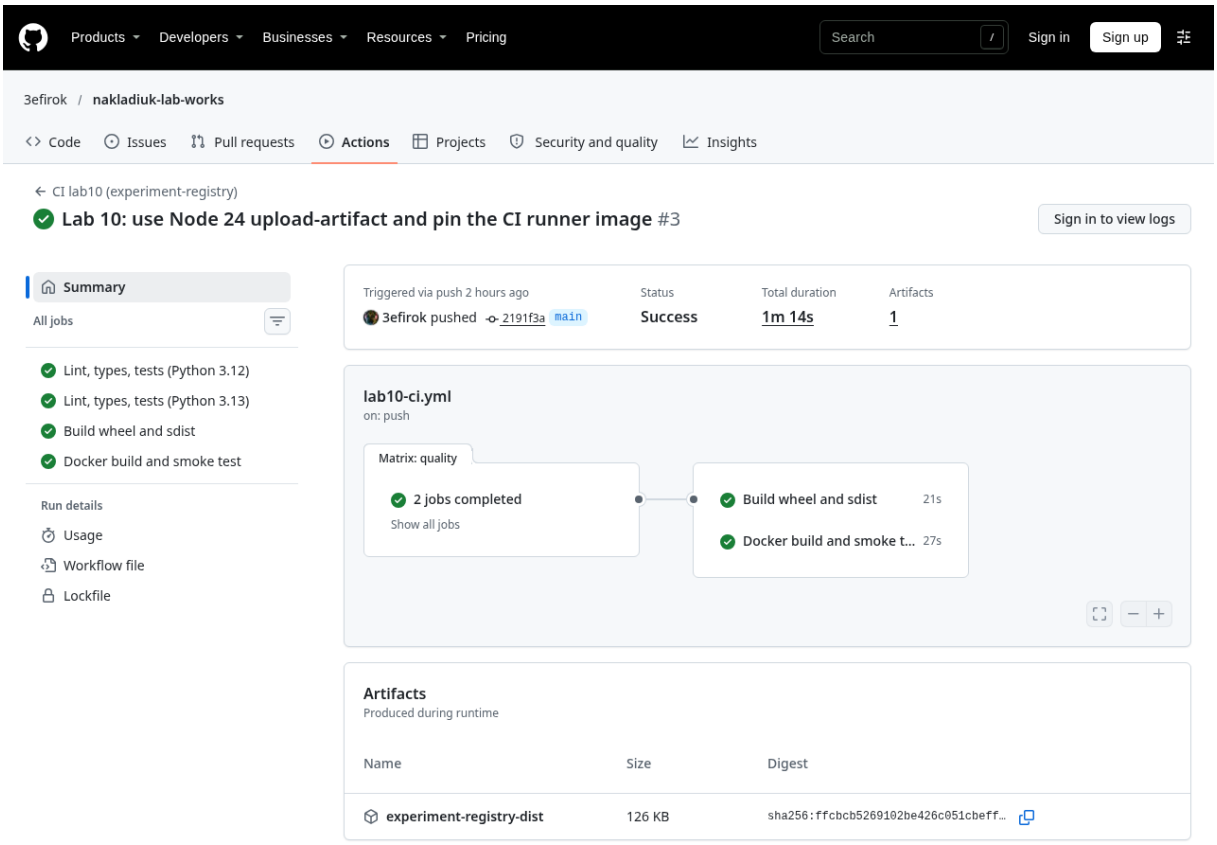


Рис. 10.18. Зелений pipeline: запуск #3 на GitHub Actions

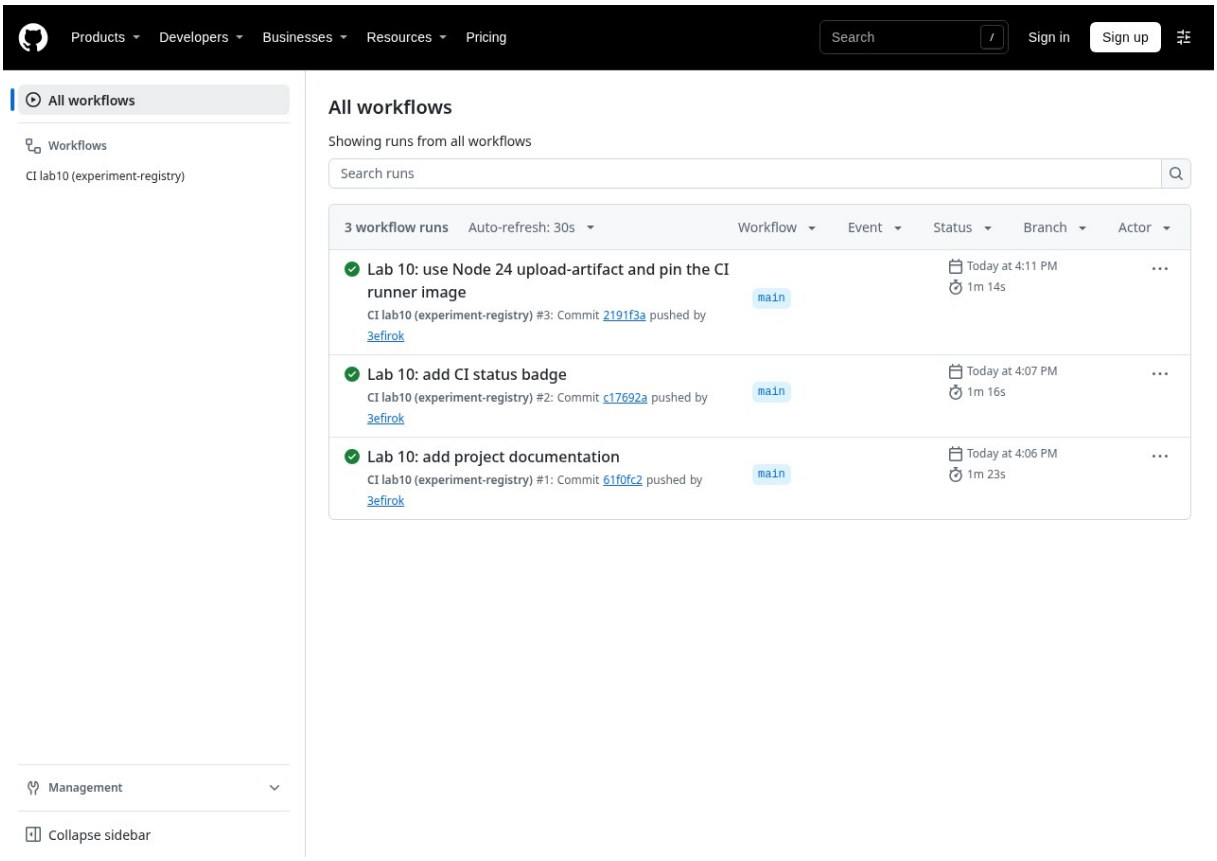


Рис. 10.19. Сторінка Actions репозиторію

4.12. README

Лістинг 10.11. README.md (фрагмент)

```
# Experiment Registry

[![CI lab10](https://github.com/3efirok/nakladiuk-lab-works/actions/workflows/lab10-ci.yml/badge.svg)]
(https://github.com/3efirok/nakladiuk-lab-works/actions/workflows/lab10-ci.yml)

Production-версія навчального проєкту з дисципліни «Професійний Пайтон» (лабораторна робота №10).
Варіант 15 — облік результатів наукових експериментів: назва експерименту, дослідник, параметр,
результат і дата.

Пакет: `experiment-registry` (модуль `experiment_registry`), версія 1.0.0.

## Можливості

- додавання дослідників та експериментів, пошук за дослідником, average / minimum / maximum,
  фільтрація за діапазоном, сортування за датою;
- REST API на FastAPI з Pydantic-валідацією та Swagger UI (`/docs`);
- persistence layer: SQLite + SQLAlchemy 2.0, міграції Alembic;
- імпорт CSV / експорт JSON з YAML-конфігурацією, потокова обробка великих файлів;
- оптимізована статистика (NumPy + кеш з інвалідацією), паралельні запити через HTTPX;
- `GET /health` для Docker HEALTHCHECK.

## Встановлення (локально)

```bash
python -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[dev]"
cp .env.example .env            # за потреби змінити значення
alembic upgrade head
experiment-registry serve       # http://127.0.0.1:8000/docs
```

Конфігурація

Усі налаштування читаються зі змінних середовища (або з `.env`, який не потрапляє в Git):

Змінна	За замовчуванням	Опис
`DATABASE_URL`	`sqlite:///experiments.db`	URL бази даних SQLAlchemy
`ENVIRONMENT` (`APP_ENV`)	`development`	`development`, `test`, `production`
`LOG_LEVEL`	`INFO`	рівень журналювання
`API_TOKEN`	не задано	якщо задано, POST/PATCH/DELETE потребують заголовка `X-API-Token`

Справжні паролі, токени й ключі не зберігаються в коді, README, Dockerfile чи Git; приклад
змінних — у `.env.example`.

API

Method	Endpoint	Опис
GET	`/health`	стан застосунку та бази
GET, POST	`/researchers`	список / додавання дослідників
GET	`/researchers/{id}`, `/researchers/{id}/experiments`	дослідник і його експерименти
GET, POST	`/experiments`	список (фільтри `researcher_id`, `parameter`, `min_result`,
`max_result`, `sort`, `limit`, `offset`) / додавання
GET, PATCH, DELETE	`/experiments/{id}`	експеримент: читання, зміна, видалення
GET	`/experiments/statistics`	average, minimum, maximum (SQL)
GET	`/experiments/statistics/extended`	mean, std, median, p95 (NumPy, кеш)
GET	`/experiments/statistics/by-researcher?ids=1&ids=2`	паралельна статистика кількох дослідників

Docker

```bash
docker build -t experiment-registry:1.0.0 .
```
```

4.13. Git commits

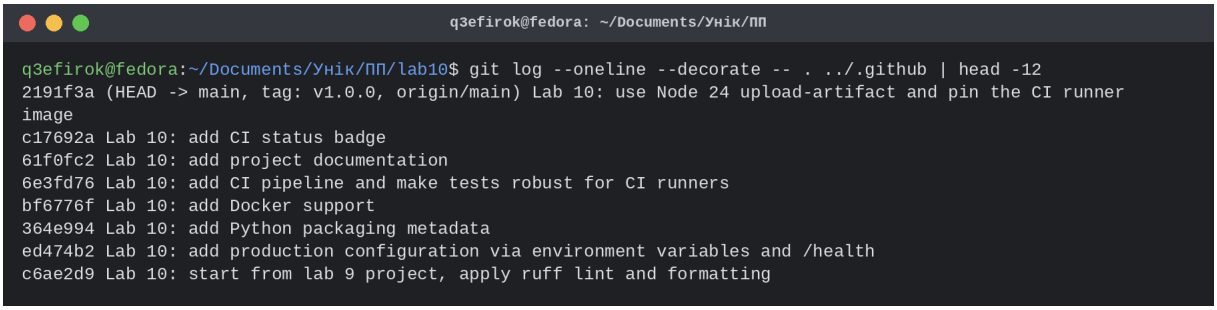


Рис. 10.20. Історія комітів лабораторної №10

4.14. Production checklist

Таблиця 10.1. Production checklist

Пункт	Стан	Підтвердження
проєкт запускається локально	✓	`experiment-registry serve`, Swagger UI
tests проходять	✓	245 passed, coverage 92.88%
lint і format проходять	✓	ruff check, ruff format --check
type checking проходить	✓	mypy --strict
wheel і sdist створюються	✓	`python -m build`
wheel встановлюється	✓	чистий venv, `experiment-registry --help`
.env відсутній у Git, .env.example є	✓	`git check-ignore`, .env.example
secrets відсутні у коді	✓	`SecretStr`, лише змінні середовища
Docker image створюється	✓	`docker build`, multi-stage
container запускається, /health → 200	✓	healthy, `{"status": "ok"}`
database працює, дані зберігаються	✓	міграції при старті, том `/data`
REST API працює	✓	curl-запити, Swagger UI
README актуальний	✓	встановлення, конфігурація, Docker, API, CI
CI pipeline green	✓	GitHub Actions run #3
Git history зрозуміла, version визначена	✓	логічні коміти, 1.0.0, тег v1.0.0

4.15. Завдання підвищеної складності

- multi-stage Docker build, non-root Docker user, Docker HEALTHCHECK;
- автоматичні migrations при старті контейнера;
- coverage у CI з порогом 80% (`fail_under`) і матриця Python 3.12/3.13;
- upload wheel artifact у CI, перевірка встановлення wheel;
- Docker image tagging (`experiment-registry:1.0.0`), semantic versioning, git tag v1.0.0;
- захист операцій запису токеном через `SecretStr` і заголовок `X-API-Token`.

5. Відповіді на контрольні запитання

1. Що таке Python package?

Python package (import package) - це каталог із модулями, який імпортується як одне ціле і зазвичай містить файл __init__.py. У проєкті це пакет experiment_registry, розміщений у src/experiment_registry відповідно до src-layout.

2. Що таке distribution package?

Distribution package - це архів для поширення та встановлення (wheel або sdist), який містить один або кілька import packages і metadata: назву, версію, залежності. Ім'я distribution може відрізнятися від імені import package: у проєкті встановлюється experiment-registry, а імпортується experiment_registry.

3. Що таке wheel?

Wheel - стандартний built-формат distribution package (PEP 427): zip-архів із готовим до встановлення кодом і metadata, який pip просто розпаковує без кроку збирання. Тому встановлення з wheel швидше й передбачуваніше, ніж зі sdist.

4. Яке розширення має wheel?

Wheel має розширення .whl; ім'я файлу містить назву, версію і теги версії Python, ABI та платформи. Для pure-Python пакета проєкту воно має вигляд experiment_registry-<версія>-py3-none-any.whl.

5. Що таке sdist?

Sdist (source distribution) - архів .tar.gz із вихідним кодом, pyproject.toml і metadata, з якого pip під час встановлення спочатку збирає wheel. Він потрібен для аудиту коду та збирання на платформах, для яких немає готового wheel.

6. Для чого використовується pyproject.toml?

pyproject.toml - стандартний конфігураційний файл Python-проєкту (PEP 518, PEP 621), що описує build system, metadata і залежності пакета, а також налаштування інструментів у секціях [tool.*], наприклад ruff, mypy, pytest і coverage. У проєкті саме з нього python -m build збирає wheel і sdist.

7. Що містить [build-system]?

[build-system] містить requires - список пакетів, потрібних для збирання (наприклад, setuptools>=70), і build-backend - об'єкт, який виконує збирання (наприклад, setuptools.build_meta). Build frontend (pip або python -m build) встановлює ці залежності в ізольоване середовище і викликає backend.

8. Що містить [project]?

[project] містить metadata пакета за PEP 621: name, version, description, readme, requires-python, dependencies, license, authors, а також optional-dependencies (наприклад, група dev) і scripts для консольних команд. У проєкті там вказано name = experiment-registry та runtime-залежності, зокрема FastAPI, SQLAlchemy і pydantic-settings.

9. Як створити wheel?

Потрібно встановити пакет build (python -m pip install build) і виконати python -m build --wheel у корені проєкту; файл .whl з'явиться в каталозі dist/. У проєкті

зібраний wheel потім встановлювався в чистий venv, щоб перевірити, що пакет працює поза робочим каталогом.

10. Як створити sdist?

Sdist створюється командою python -m build --sdist у корені проєкту, результат - архів dist/experiment_registry-<версія>.tar.gz. Команда python -m build без прапорців створює і sdist, і wheel.

11. Для чого використовується python -m build?

python -m build запускає стандартний build frontend за PEP 517: створює ізольоване віртуальне середовище, встановлює залежності з [build-system] і викликає build backend. Без аргументів він збирає sdist, потім із нього wheel і кладе обидва в dist/; у проєкті ця команда виконується і локально, і в CI.

12. Що таке semantic versioning?

Semantic versioning (SemVer) - схема версіонування MAJOR.MINOR.PATCH, у якій номер версії повідомляє, наскільки зміни сумісні з попереднім публічним API. Наприклад, перехід з 1.4.2 на 2.0.0 означає несумісні зміни, а на 1.4.3 - лише виправлення помилок.

13. Що означає MAJOR?

MAJOR збільшується при несумісних (breaking) змінах публічного API, наприклад видаленні чи перейменуванні endpoint або зміні формату відповіді. При цьому MINOR і PATCH скидаються до 0.

14. Що означає MINOR?

MINOR збільшується при додаванні нової функціональності, сумісної з попередніми версіями, наприклад нового endpoint статистики за researcher. При цьому PATCH скидається до 0.

15. Що означає PATCH?

PATCH збільшується при сумісних виправленнях помилок без зміни API, наприклад виправленні обчислення стандартного відхилення або валідації вхідних даних.

16. Що таке environment variable?

Environment variable - іменована пара ключ=значення в середовищі процесу ОС, яку процес успадковує від батьківського і може прочитати через os.environ або os.getenv. У проєкті змінні DATABASE_URL, ENVIRONMENT і LOG_LEVEL автоматично читає клас Settings на основі pydantic-settings.

17. Навіщо configuration виносити з коду?

Щоб один і той самий код і Docker image працювали в різних середовищах (development, test, production) лише зі зміною параметрів, без редагування коду. Це також не дає secrets потрапити в репозиторій і відповідає принципам 12-factor app; у проєкті адреса бази даних задається через DATABASE_URL, а не жорстко в коді.

18. Що таке secret?

Secret - конфіденційне значення, розкриття якого дає несанкціонований доступ до систем або даних. Secrets передаються застосунку лише під час виконання: через

environment variables, файл .env поза Git або сховища секретів, наприклад GitHub Secrets.

19. Наведіть приклади secrets.

Паролі до баз даних і connection strings з обліковими даними, API keys, access tokens (наприклад, токени PyPI або Docker Hub), приватні SSH- і TLS-ключі, ключі підпису JWT, OAuth client secrets.

20. Чому secrets не можна commit у Git?

Git зберігає всю історію, тому секрет залишається доступним навіть після видалення з поточного коміту, а репозиторій можуть клонувати, форкнути або зробити публічним. Боти автоматично сканують GitHub у пошуках ключів, тому скомпрометований секрет потрібно негайно відкликати й замінити (rotate).

21. Для чого використовується .env?

Файл .env містить пари KEY=value для локальної розробки, з яких pydantic-settings (параметр env_file) або docker run --env-file завантажують конфігурацію. У проєкті в ньому локально задаються DATABASE_URL, ENVIRONMENT і LOG_LEVEL.

22. Чому .env має бути у .gitignore?

Тому що .env містить реальні локальні значення та secrets конкретного розробника або середовища. Якщо його закомітити, секрети потраплять в історію Git, тому в проєкті .env ігнорується, а в репозиторії зберігається лише .env.example.

23. Для чого потрібний .env.example?

.env.example - це шаблон, що документує всі потрібні змінні середовища з безпечними або порожніми значеннями, без реальних secrets. Розробник копіює його в .env і заповнює своїми значеннями; у проєкті він містить DATABASE_URL, ENVIRONMENT і LOG_LEVEL.

24. Що таке Docker?

Docker - платформа контейнеризації, яка пакує застосунок разом із runtime, залежностями й налаштуваннями в image і запускає його в ізольованому container. Контейнери використовують ядро хост-системи (namespaces, cgroups), тому легші за віртуальні машини і забезпечують однакову поведінку на різних машинах.

25. Що таке Dockerfile?

Dockerfile - текстовий файл з інструкціями (FROM, WORKDIR, COPY, RUN, ENV, EXPOSE, USER, HEALTHCHECK, CMD), за якими docker build крок за кроком створює image; інструкції, що змінюють файлову систему, створюють шари (layers), які кешуються. У проєкті це multi-stage Dockerfile з non-root user і HEALTHCHECK.

26. Що таке Docker image?

Docker image - незмінний (read-only) шаблон із шарів, що містить базову ОС, Python, залежності, код застосунку і metadata, зокрема команду запуску. Image ідентифікується ім'ям і тегом (name:tag), і з нього можна запустити будь-яку кількість containers.

27. Що таке Docker container?

Docker container - запущений ізольований екземпляр image з власними процесами, файловою системою (тонкий записуваний шар поверх image), мережею та змінними середовища. Після видалення контейнера дані з його записуваного шару втрачаються, якщо не використовувати volume.

28. Чим image відрізняється від container?

Image - статичний незмінний шаблон, а container - його запущений екземпляр зі станом і життєвим циклом (created, running, stopped). Співвідношення схоже на клас і об'єкт: з одного image можна запустити кілька незалежних containers.

29. Для чого використовується docker build?

docker build створює image з Dockerfile і build context (зазвичай поточний каталог), наприклад docker build -t <name>:<tag> . , де -t задає ім'я та тег. Кеш шарів пришвидшує повторні збирання, а в CI проєкту docker build перевіряє, що image коректно збирається.

30. Для чого використовується docker run?

docker run створює і запускає container з image; основні параметри: -p 8000:8000 публікує порт, -e або --env-file передають змінні середовища, -v підключає volume, --rm видаляє контейнер після зупинки, -d запускає у фоні. У проєкті так запускається API з volume для SQLite-бази.

31. Що робить EXPOSE?

EXPOSE документує, на якому порту застосунок у контейнері приймає з'єднання (наприклад, EXPOSE 8000 для uvicorn), і зберігає це в metadata image. Сам він порт назовні не відкриває: для цього потрібен docker run -p 8000:8000.

32. Що робить CMD?

CMD задає команду за замовчуванням, яка виконується при старті контейнера, наприклад запуск uvicorn з FastAPI-застосунком на 0.0.0.0:8000. Рекомендована exec-форма (JSON-масив), щоб процес коректно отримував сигнали; CMD можна перевизначити аргументами docker run, і діє лише останній CMD у Dockerfile.

33. Для чого потрібний WORKDIR?

WORKDIR встановлює робочий каталог для наступних інструкцій RUN, COPY, CMD і для процесу запущеного контейнера, створюючи каталог за потреби, наприклад WORKDIR /app. Це дозволяє використовувати відносні шляхи і не писати cd у RUN.

34. Що робить COPY?

COPY копіює файли й каталоги з build context у файлову систему image, наприклад COPY src ./src, а в multi-stage build COPY --from=<stage> копіює артефакти з попереднього етапу. Спочатку зазвичай копіюють файли залежностей, а потім код, щоб ефективніше використовувати кеш шарів.

35. Для чого використовується .dockerignore?

.dockerignore визначає файли й каталоги, які не передаються в build context: .git, .venv, __pycache__, кеші pytest, mypy і ruff, dist і, головне, .env. Це зменшує контекст і image, пришвидшує збирання і не дає secrets потрапити в image.

36. Що таке Docker volume?

Docker volume - керований Docker механізм постійного зберігання даних поза записуваним шаром контейнера, дані якого не залежать від життєвого циклу контейнера. У проєкті файл SQLite-бази зберігається у volume, тому результати експериментів не зникають після перезапуску або видалення контейнера.

37. Для чого потрібний health endpoint?

Health endpoint - легкий endpoint (у проєкті GET /health), що повертає статус застосунку, наприклад {"status": "ok"} з кодом 200, якщо сервіс працює. Його використовують Docker HEALTHCHECK, оркестратори, load balancers і системи моніторингу, щоб автоматично виявляти несправні екземпляри.

38. Що таке CI?

CI (Continuous Integration) - практика частої інтеграції змін у спільний репозиторій, за якої кожен push і pull request автоматично проходить перевірки: lint, type checking, tests і build. Це дозволяє виявляти помилки рано, поки зміни ще невеликі.

39. Що таке CD?

CD - це Continuous Delivery, тобто автоматична підготовка перевірених артефактів (wheel, Docker image, release) до випуску, коли розгортання в production залишається ручним рішенням. Continuous Deployment іде далі й автоматично розгортає кожну зміну, що пройшла pipeline.

40. Що таке GitHub Actions?

GitHub Actions - вбудована в GitHub CI/CD-платформа, що запускає автоматизовані workflows у відповідь на події репозиторію: push, pull_request, розклад або ручний запуск. У проєкті workflow виконує ruff check, ruff format --check, mypy, pytest з coverage, python -m build і docker build.

41. Що таке workflow?

Workflow - автоматизований процес, описаний YAML-файлом у каталозі .github/workflows/, який визначає тригери (on) та одну або кілька jobs. Кожен запуск workflow (workflow run) завершується статусом success або failure.

42. Що таке job?

Job - набір steps, які послідовно виконуються на одному runner у спільному середовищі; jobs одного workflow за замовчуванням виконуються паралельно, а залежності між ними задаються через needs. Кожна job вказує runs-on, наприклад ubuntu-latest, і може використовувати matrix для кількох версій Python.

43. Що таке step?

Step - окремий крок job: shell-команда (run:) або готова action (uses:), наприклад actions/checkout чи actions/setup-python. Steps виконуються послідовно, і якщо step

завершується з ненульовим кодом виходу, job за замовчуванням вважається провaленою.

44. Що таке runner?

Runner - машина (віртуальна машина або контейнер), яка виконує jobs workflow: GitHub-hosted (ubuntu-latest, windows-latest, macos-latest) або self-hosted. GitHub-hosted runner для кожної job надає чисте середовище, тому залежності встановлюються щоразу заново.

45. Коли повинен запускатися CI?

CI має запускатися автоматично при кожному push і кожному pull request, щоб перевірити зміни до злиття в main; додатково його можна запускати за розкладом (schedule) або вручну (workflow_dispatch). Злиття pull request доцільно дозволяти лише після green pipeline.

46. Для чого потрібний lint?

Lint (linting) - статичний аналіз коду без його запуску, що виявляє невикористані імпорти й змінні, невизначені імена, потенційні bugs і порушення стилю. Він підтримує єдиний стиль коду і дешево знаходить помилки ще до тестів і code review.

47. Для чого використовується Ruff?

Ruff - дуже швидкий лінтер і форматер Python, написаний на Rust, який замінює flake8, isort, pyupgrade, black та інші інструменти. У CI проєкту ruff check . перевіряє правила лінтингу, а ruff format --check . перевіряє форматування без зміни файлів.

48. Для чого потрібний mypy?

mypy - статичний type checker для Python, який за type hints перевіряє узгодженість типів без запуску програми, наприклад передачу str замість float або використання значення, що може бути None. У CI проєкту mypy перевіряє код пакета experiment_registry і зупиняє pipeline при помилках типів.

49. Що таке type checking?

Type checking - перевірка відповідності типів значень оголошеним типам. Інтерпретатор Python анотації під час виконання не перевіряє, тому статичну перевірку виконують інструменти на кшталт mypy, а runtime-валідацію вхідних даних API у FastAPI забезпечує Pydantic.

50. Навіщо tests запускати в CI?

Щоб кожна зміна автоматично перевірялася в чистому, відтворюваному середовищі, незалежно від машини розробника, і регресії виявлялися до злиття в main. У проєкті pytest із coverage запускається в CI, і будь-який непройдений тест робить pipeline червоним.

51. Навіщо package build виконувати в CI?

Щоб переконатися, що pyproject.toml коректний і python -m build справді створює wheel та sdist з усіма модулями й metadata. Інакше помилки пакування (відсутні файли, неправильні залежності чи конфігурація src-layout) виявляються лише під час встановлення в іншому середовищі.

52. Навіщо Docker build виконувати у CI?

Щоб перевірити, що Dockerfile коректний, потрібні файли не відфільтровано .dockerignore, а залежності успішно встановлюються в image. Це гарантує, що кожен коміт, який пройшов CI, можна контейнеризувати й розгорнути.

53. Що таке GitHub Secret?

GitHub Secret - зашифроване значення, що зберігається в налаштуваннях репозиторію, environment або організації (Settings, Secrets and variables, Actions) і доступне workflow лише під час виконання. У коді його значення відсутнє, а в логах GitHub автоматично маскує його як ***.

54. Як використовувати secret у workflow?

До secret звертаються через вираз ${{ secrets.NAME }}, зазвичай передаючи його в змінну середовища step, наприклад env: API_TOKEN: ${{ secrets.API_TOKEN }}. Значення не можна записувати прямо в YAML чи виводити в логи; для pull request із форків secrets за замовчуванням недоступні.

55. Що таке documentation?

Documentation - сукупність матеріалів, що пояснюють призначення, встановлення, налаштування, використання і будову проєкту: README, docstrings, автоматична OpenAPI-документація, коментарі, changelog. Вона потрібна користувачам, новим розробникам і для захисту проєкту.

56. Що повинен містити README?

README повинен містити опис предметної системи (облік результатів наукових експериментів), вимоги, installation, configuration через environment variables і .env.example, запуск локально та в Docker, опис API з посиланням на /docs, запуск tests, опис CI/CD, структуру проєкту і сценарій демонстрації. Реальних secrets у README бути не повинно.

57. Для чого потрібні docstrings?

Docstrings - рядки документації на початку модуля, класу або функції, доступні під час виконання через __doc__ і help(). Вони описують призначення, параметри, повернене значення та винятки, їх використовують IDE і генератори документації, а FastAPI бере docstring endpoint як його опис в OpenAPI.

58. Що таке OpenAPI?

OpenAPI - стандартна мовонезалежна специфікація опису REST API у форматі JSON або YAML: endpoints, HTTP-методи, параметри, схеми запитів і відповідей, коди статусів, автентифікація. На її основі будуються інтерактивна документація, генератори клієнтів і інструменти тестування.

59. Що таке /docs у FastAPI?

/docs - сторінка інтерактивної документації Swagger UI, яку FastAPI автоматично генерує з OpenAPI-схеми застосунку. На ній можна переглянути всі endpoints API проєкту та виконати запити безпосередньо з браузера; альтернативна документація ReDoc доступна на /redoc.

60. Що таке /openapi.json?

/openapi.json - endpoint, за яким FastAPI віддає автоматично згенеровану OpenAPI-специфікацію API у форматі JSON, побудовану з маршрутів, type hints і Pydantic-моделей. Саме її використовують Swagger UI на /docs, ReDoc і генератори клієнтів.

61. Що таке logging?

Logging - запис подій роботи застосунку (інформація, попередження, помилки) з рівнем важливості, часом і джерелом за допомогою стандартного модуля logging. У проєкті рівень логування задається змінною LOG_LEVEL, наприклад DEBUG, INFO, WARNING або ERROR.

62. Чому logging краще за print?

logging має рівні важливості, які змінюються конфігурацією без зміни коду, форматування з часом і назвою модуля, різні handlers (консоль, файл, системи збору логів) і запис traceback через logger.exception(). print лише виводить текст у stdout без рівнів і контексту, тому його важко фільтрувати й аналізувати в production.

63. Що таке production configuration?

Production configuration - набір налаштувань для реальної експлуатації: ENVIRONMENT=production, рівень логування INFO або WARNING, вимкнені debug і autoreload, шлях до бази даних у persistent volume, secrets лише із середовища. У проєкті ці значення передаються в контейнер через environment variables, а не змінюються в коді.

64. Чим development відрізняється від production?

Development орієнтований на зручність розробки: autoreload (uvicorn --reload), детальне логування DEBUG, локальна SQLite-база і тестові дані. Production орієнтований на стабільність, безпеку й продуктивність: без reload і debug, рівень INFO або WARNING, secrets із середовища, persistent volume, non-root user і healthcheck.

65. Що таке Docker healthcheck?

Docker healthcheck - інструкція HEALTHCHECK у Dockerfile, яка задає команду, що Docker періодично виконує всередині контейнера (параметри --interval, --timeout, --retries); за її кодом виходу контейнер отримує статус healthy або unhealthy. У проєкті healthcheck викликає GET /health, тому статус показує, що API справді відповідає, а не лише що процес живий.

66. Для чого потрібний non-root user?

За замовчуванням процеси в контейнері працюють від root, тому при вразливості застосунку зловмисник отримує максимальні права в контейнері та більше можливостей вийти за його межі. Тому в Dockerfile проєкту створюється непривілейований користувач і вмикається інструкцією USER, що реалізує принцип найменших привілеїв.

67. Що таке multi-stage build?

Multi-stage build - Dockerfile з кількома інструкціями FROM, де на першому етапі (builder) встановлюються інструменти збирання і збирається застосунок, а

фінальний етап копіює лише потрібні артефакти через COPY --from. У результаті production image менший і містить менше пакетів і потенційних вразливостей.

68. Що таке coverage?

Coverage (test coverage) - метрика, яка показує, яка частка рядків або гілок коду виконувалася під час тестів; вимірюється через coverage.py або pytest-cov, наприклад pytest --cov=experiment_registry --cov-report=term-missing. Висока coverage не гарантує правильності, але низька вказує на неперевірені частини коду.

69. Навіщо встановлювати coverage threshold?

Coverage threshold (наприклад, --cov-fail-under=80 або fail_under у конфігурації coverage) змушує CI завершуватися з помилкою, якщо покриття опускається нижче порогу. Так зміни без тестів не проходять перевірку, і якість тестування не деградує поступово.

70. Що означає green CI pipeline?

Green CI pipeline означає, що всі jobs і steps workflow завершилися успішно і GitHub показує зелену позначку: у проєкті це ruff check, ruff format --check, mypy, pytest з coverage, python -m build і docker build. Лише такий коміт доцільно вважати готовим до злиття та release.

6. Висновок

У лабораторній роботі проєкт experiment-registry доведено до production-oriented стану. Пакет версії 1.0.0 має повні метадані в pyproject.toml і збирається у wheel та sdist, а wheel встановлюється в чистому середовищі. Конфігурація (`DATABASE_URL`, `APP_ENV`, `LOG_LEVEL`) передається через змінні середовища за допомогою pydantic-settings, секрети не зберігаються в коді та Git, а для перевірки стану додано `GET /health`.

Багатоетапний Docker image запускається від непривілейованого користувача, застосовує міграції, зберігає SQLite у томі й проходить HEALTHCHECK. GitHub Actions автоматично виконує ruff, mypy, 245 тестів із покриттям 92.88%, збирання пакета і Docker-образу — pipeline зелений. Проєкт, що розвивався з першої лабораторної роботи, тепер відтворюваний, протестований, конфігурований, контейнеризований і готовий до демонстрації.